华中科技大学
机械科学与工程学院

项目设计报告

工程项目名称——二轮平衡小车系统设计

姓　名：郭沫晗，郑浩东

学　号：U202515287，U202515279

班　级：机器人科创 2502 班

指导老师：王峥

日　期：2026 年 6 月

学生 (或项目小组全体成员) 与项目作品合影照片

（粘贴处）

目 录

第一章 项目概述与目标 **1**

1.1 项目背景 1

1.2 基础设计目标 1

1.3 扩展设计目标 2

1.4 报告整体结构说明 2

第二章 项目方案 **3**

2.1 整体设计思路 3

2.2 系统工作原理 3

2.3 方案选型论证 4

2.3.1 主控芯片：STM32F103C8T6 4

2.3.2 姿态传感器：MPU6050 + DMP 4

2.3.3 电机驱动：TB6612 5

2.3.4 控制算法：串级 PID 5

第三章 系统硬件及软件架构 **6**

3.1 系统硬件架构 6

3.1.1 硬件整体框图 6

3.1.2 各模块详细说明 6

3.1.3 硬件引脚映射表 8

3.2 系统软件架构 9

3.2.1 软件设计思想 9

3.2.2 主程序总流程图 10

3.2.3 软件分层结构 11

3.2.4 核心中断流程图 12

3.2.5 系统初始化流程 13

第四章 关键技术及程序设计说明 15

4.1 核心关键技术 15

4.1.1 姿态解算技术 15

4.1.2 电机调速技术 15

4.1.3 编码器测速技术 16

4.1.4 闭环控制技术 17

4.1.5 数据滤波技术 18

4.1.6 低压保护机制 18

4.2 关键程序设计与解读 18

4.2.1 MPU6050 外部中断服务函数 18

4.2.2 直立环 PD 控制函数 19

4.2.3 速度环增量 PI 控制函数 20

4.2.4 定时器编码器模式配置 20

4.2.5 电机驱动输出函数 21

4.2.6 电压采集与滤波函数 22

第五章 系统调试过程 23

5.1 分模块调试 23

5.1.1 基础外设调试 23

5.1.2 电机驱动调试 24

5.1.3 编码器测速调试 24

5.1.4 传感器调试 24

5.1.5 电压采集调试 25

5.2 系统联调与参数整定 25

5.2.1 调参原则 25

5.2.2 直立环 PD 参数整定 25

5.2.3 速度环 PI 参数整定 26

5.2.4 功能验证 27

5.3 典型问题与解决方法 27

5.3.1 问题一：小车同向倾倒，无法直立 27

5.3.2 问题二：车体高频抖动 28

5.3.3 问题三：小车缓慢漂移 28

5.3.4 问题四：编码器方向反向 28

5.3.5 问题五：OLED 显示乱码 29

第六章 项目总结 **30**

6.1 项目完成情况 30

6.2 项目收获 30

6.2.1 硬件认知 30

6.2.2 嵌入式 HAL 库开发 31

6.2.3 控制算法理解 31

6.2.4 工程调试思维 31

6.3 不足与展望 31

6.3.1 当前局限性 31

6.3.2 后续优化方向 32

6.4 演示视频说明 32

第一章　项目概述与目标

1.1　项目背景

二轮自平衡小车是一种基于倒立摆原理的典型嵌入式控制系统。其物理本质是一阶倒立摆模型——小车质心位于轮轴上方，在无控制条件下为不稳定平衡系统，任何微小扰动都将导致倾倒。通过实时检测车体倾角并控制电机输出相应扭矩，系统可将这一天然不稳定系统转化为动态稳定系统。

该课题在工程领域具有广泛的实用价值：其核心控制算法与双足机器人、自平衡代步车（如 Segway）、灵巧型无人机等工程系统的姿态稳定原理一脉相承。在教学层面，二轮平衡小车项目涵盖了微控制器开发、传感器数据融合、PID 控制算法、电机驱动技术、嵌入式实时软件设计等多项核心知识点，是机械电子工程专业课程设计的理想载体。

本项目的硬件平台基于意法半导体（STMicroelectronics）STM32F103C8T6 微控制器，搭载 InvenSense MPU6050 六轴姿态传感器（内置 DMP 数字运动处理器），采用 TB6612 双通道 H 桥驱动芯片驱动直流减速电机，并通过正交编码器实现轮速闭环反馈，构成完整的嵌入式闭环控制系统。

1.2　基础设计目标

根据项目任务书要求，本系统须达成以下基础设计目标：

1. **直立稳定**：小车上电后能够在平坦地面自主保持直立平衡状态，稳态条件下轮向位移误差不超过 5 cm。

2. **抗干扰能力**：在承受 10° 以内摆幅的外力干扰后，系统可自主恢复至平衡状态，不出现不可逆倾倒。

3. **低压保护功能**：当电池电压经 ADC 采样换算后低于 10.0 V 时（对应 3S 锂电池单节约 3.33 V），系统自动关闭电机平衡控制输出，以保护电池与硬件电路。

1.3 扩展设计目标

在完成基础功能之上，本系统进一步实现了如下扩展功能：

1. **OLED 状态显示**：通过 0.96 英寸 OLED 屏幕实时显示俯仰角（°）、角速度（°/s）、电池电压（mV）三项关键运行参数，便于调试与运行监控。
2. **虚拟示波器（DataScope）**：基于 USART 串口通信协议，将运行时内部变量实时上传至上位机 DataScope 软件，以波形方式可视化角度、速度、电压等物理量，极大提升调试效率。
3. **独立按键启停**：通过板上独立按键实现电机输出的一键启停控制，满足安全操作需求。
4. **LED 状态指示**：以板载 LED 指示系统运行与停止状态。

上述扩展项在设计评审中对应加分评价体系——OLED 显示占附加分 5%、虚拟示波器占附加分 5%。

1.4 报告整体结构说明

本报告共分为六章：第 1 章阐述项目背景与设计目标；第 2 章论述整体方案设计与关键模块选型论证；第 3 章详细描述系统硬件架构与软件架构；第 4 章深入分析核心关键技术及其程序实现；第 5 章系统梳理分模块调试、联调与参数整定全过程；第 6 章总结项目成果、收获与展望。

第二章　项目方案

2.1　整体设计思路

本系统采用基于 STM32 微控制器的闭环反馈控制总体架构，核心设计思路如下：

1. **感知层**：通过 MPU6050 六轴传感器实时获取车体俯仰角（pitch）与角速度（gyro rate），通过正交编码器记录左右车轮的速度信息，通过 ADC 采集电池电压。
2. **决策层**：以串级 PID 控制算法为核心——内环直立环（PD 控制）以姿态角为反馈量保持车体平衡；外环速度环（PI 控制）以编码器脉冲计数为反馈量调控车速，消除稳态位置漂移。
3. **执行层**：将 PID 计算得到的 PWM 脉宽信号输出至 TB6612 电机驱动芯片，控制直流减速电机的转向与转速，产生所需平衡力矩。

系统整体控制周期为 10 ms（100 Hz），由 MPU6050 的 200 Hz 外部中断经 2 分频产生，确保控制实时性。

2.2　系统工作原理

二轮平衡小车的动力学本质为一阶倒立摆。设车体倾角为 θ（前倾为正），重力加速度为 g，等效摆长为 L，则无控制条件下车体的线性化运动方程为：

$$\ddot{\theta} = \frac{g}{L}\,\theta \tag{2.1}$$

该微分方程的特征值为正实数，表明系统开环不稳定——任何非零初始倾角 $\theta(0) \neq 0$ 均导致指数增长直至倾倒。闭环控制的核心在于：通过电机对车轮施加扭矩 T_m，产生与倾角 θ 成比例的回复力矩，将式 (2-1) 改写为：

$$\ddot{\theta} = \frac{g}{L}\,\theta - K_p\,\theta - K_d\,\dot{\theta} \tag{2.2}$$

式中 K_p 为比例增益（提供与角度偏差成比例的回复力），K_d 为微分增益（提供与角速度成比例的阻尼力）。当 $K_p > g/L$ 时，系统极点移至左半平面，车体恢复稳定。

2.3 方案选型论证

2.3.1 主控芯片：STM32F103C8T6

STM32F103C8T6 是意法半导体基于 ARM Cortex-M3 内核的 32 位微控制器，其主要资源参数与本系统需求匹配分析如表 2-1 所示。

表 2.1 STM32F103C8T6 资源匹配分析

资源项	芯片参数	本系统需求	匹配度
主频	72 MHz	$\geq$ 48 MHz	满足
Flash	64 KB	固件 $\approx$ 40 KB	满足
SRAM	20 KB	运行时 $\approx$ 8 KB	满足
定时器	4 个通用 + 2 个高级	4 路 PWM + 2 路编码器	满足
I2C 接口	2 路	MPU6050 × 1	满足
USART	3 路	调试输出 × 1	满足
ADC	2 个（12 位）	电池电压 × 1	满足
GPIO	37 个	$\approx$ 18 路	充裕

选型依据：STM32F103C8T6 在嵌入式教学实验中应用广泛，生态成熟，Keil MDK 集成开发环境支持完善，且本项目所需全部外设资源均可由该芯片提供，无额外扩展需求。

2.3.2 姿态传感器：MPU6050 + DMP

MPU6050 是 InvenSense 公司的六轴运动传感器，内部集成三轴陀螺仪、三轴加速度计及一个可编程的数字运动处理器（Digital Motion Processor, DMP）。相比分立传感器方案，其核心优势在于：

1. **DMP 硬件姿态解算**：DMP 可直接输出四元数/欧拉角，将复杂的姿态解算（Madgwick / Mahony 滤波算法）从 MCU 卸载到传感器硬件，显著降低 MCU 运算负荷。

2. **单芯片集成**：陀螺仪与加速度计已内部校准，避免分立器件间的轴间误差。

3. **200 Hz 中断输出**：INT 引脚可配置为数据就绪中断，天然适配本系统的 10 ms 定时控制周期。

2.3.3 电机驱动：TB6612

TB6612FNG 是东芝半导体的双通道 H 桥电机驱动芯片，主要性能参数为：单通道持续输出电流 1.2 A（峰值 3.2 A），支持 PWM 频率最高 100 kHz，内置低导通电阻 MOSFET（0.5 Ω），待机电流小于 1 μA。其控制逻辑简洁——每个电机仅需 3 个控制引脚（PWM + IN1 + IN2），与 L298N 相比体积更小、效率更高、外接元件更少。

2.3.4 控制算法：串级 PID

本系统采用串级 PID 方案（内环直立 PD + 外环速度 PI），其选型理由如下：

1. **串级结构的鲁棒性**：内环（直立环）以最高优先级维持车体姿态稳定，外环（速度环）通过调节内环的期望倾角间接控制车速。两环解耦良好，各自参数独立可调。

2. **PD 直立环的物理合理性**：比例项 $K_p\theta$ 提供与倾角成比例的回复力，微分项 $K_d\dot{\theta}$ 提供与角速度成比例的阻尼力——二者共同构成机械弹簧-阻尼系统的电学等效，物理意义明确。

3. **PI 速度环消静差**：积分项 $\int e\,dt$ 持续累加速度偏差，直至稳态误差为零，解决纯比例控制下小车缓慢漂移的问题。

第三章　系统硬件及软件架构

3.1　系统硬件架构

3.1.1　硬件整体框图

系统硬件由五大功能单元构成：主控单元（STM32F103 最小系统）、电机驱动单元（TB6612 + 直流减速电机 + 正交编码器）、姿态传感单元（MPU6050）、电源单元（3S 锂电池 + 稳压电路 + 分压采集）、人机交互单元（OLED 显示屏 + 独立按键 + LED 指示灯 + USART 串口）。各单元连接关系如图 3.1 所示。

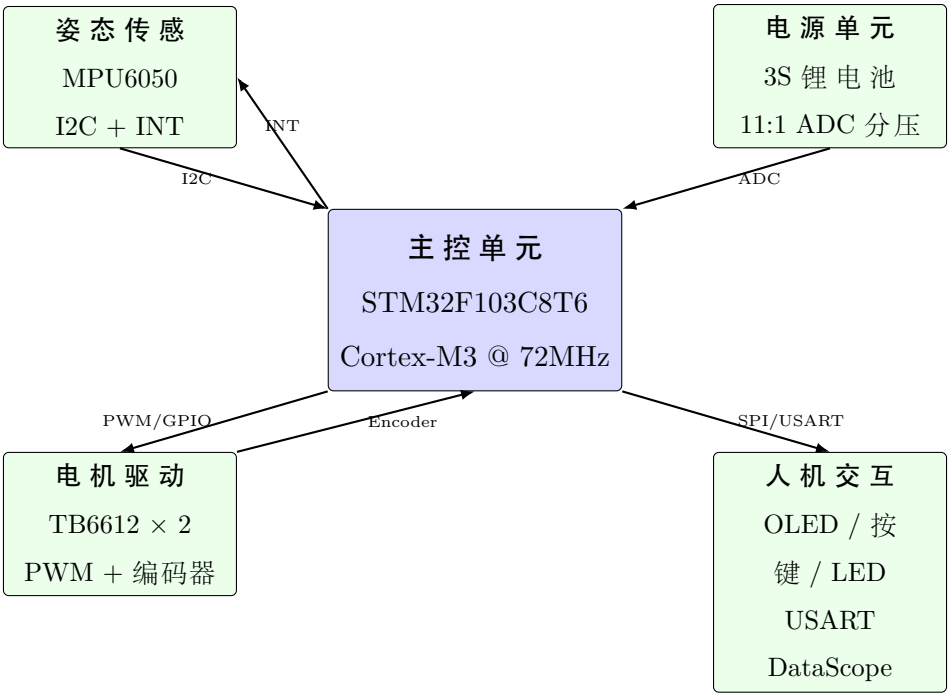


图 3.1　系统硬件架构框图

3.1.2　各模块详细说明

主控模块

STM32F103C8T6 最小系统包含：8 MHz 外部晶振（经内部 PLL 倍频至 72 MHz）、3.3 V 稳压供电（AMS1117-3.3）、SWD 调试接口（ST-Link / J-Link）、复位电路。片上

外设资源配置汇总如表 3.1 所示。

表 3.1 片上外设资源分配总表

外设	通道	功能	关联引脚
TIM1	CH1 + CH4	双路 PWM 输出（电机 A / B）	PA8、PA11
TIM3	CH1 + CH2	左轮编码器 AB 相计数	PA6、PA7
TIM4	CH1 + CH2	右轮编码器 AB 相计数	PB6、PB7
GPIO	SCL + SDA	MPU6050 通信（软件 I2C，地址 0x68）	PB8、PB9
ADC1	CH8	电池电压采集（12 位）	PB0
USART1	TX + RX	调试输出与 DataScope	PA9、PA10
EXTI9-5	PA5	KEY0 独立按键中断	PA5
EXTI15-10	PA12	MPU6050 INT 中断（200 Hz）	PA12
GPIO	—	OLED SPI 软件模拟 × 4	PB3/PB4/PB5/PA1/PA15
GPIO	—	电机方向控制 IN1/IN2 × 4	PB12–PB15
GPIO	—	状态 LED	PA4

电机与驱动模块

电机选用直流减速电机，自带 AB 相正交编码器。编码器每转输出固定脉冲数（典型值 390 PPR，经 4 倍频后为 1560 计数/转），通过 TIM3 / TIM4 的编码器模式自动对 AB 相脉冲计数，无需 CPU 干预。

TB6612 电机驱动逻辑如表 3.2 所示，每个电机需 3 个控制信号。

表 3.2 TB6612 电机控制逻辑

IN1	IN2	电机状态
1	0	正转（前进）
0	1	反转（后退）
1	1	短接制动
0	0	滑行停止

姿态传感模块

MPU6050 通过软件 I2C 接口与 MCU 通信，SCL 为 PB8、SDA 为 PB9，器件地址为 0x68。AD0 在模块端接地，固件不再驱动 PA15，避免与 OLED 的 RS 引脚冲突。

INT 中断引脚连接至 PA12，配置为下降沿触发，MPU6050 DMP 数据就绪时以 200 Hz 频率产生中断信号。该中断作为系统控制周期的时基来源。

电源模块

系统采用 3S 锂电池供电（标称电压 11.1 V，满电约 12.6 V）。电池电压经 11:1 电阻分压网络（R1 = 100 kΩ / R2 = 10 kΩ）衰减后，送入 STM32 的 ADC1_CH8（PB0）进行 12 位模数转换。换算公式为：

$$V_{\text{bat}} = \frac{V_{\text{ADC}}}{2^{12}} \times 3.3 \times \frac{R_1 + R_2}{R_2} = \frac{V_{\text{ADC}}}{4096} \times 3.3 \times 11 \tag{3.1}$$

式中 V_{ADC} 为 ADC 原始数值，3.3 V 为 ADC 参考电压。

人机交互模块

- **OLED 显示屏**：0.96 英寸 SSD1306 驱动，分辨率 128×64，采用 4 线 SPI 软件模拟接口（PB3-RST、PA1-CS、PA15-D/C、PB5-SCLK、PB4-SDIN），实时刷新显示俯仰角（pitAngle）、角速度（pitVel）、电池电压（batVol）三个关键参数。
- **独立按键**：KEY0 连接 PA5，下降沿触发 EXTI 中断，实现电机输出启停切换。
- **状态 LED**：PA4 驱动，亮起表示电机运行中，熄灭表示已停机。
- **USART 串口**：USART1（PA9-TX / PA10-RX）经 CH340 USB 转串口芯片连接至上位机，实现 printf 调试输出与 DataScope 虚拟示波器数据上传。

3.1.3 硬件引脚映射表

完整引脚映射关系如表 3.3 所示。

表 3.3 系统硬件引脚映射总表

功能	引脚	GPIO	外设/模式
电机 A PWM	PA8	TIM1_CH1	高级定时器 PWM
电机 B PWM	PA11	TIM1_CH4	高级定时器 PWM
电机 A IN1	PB14	GPIO	推挽输出
电机 A IN2	PB15	GPIO	推挽输出
电机 B IN1	PB13	GPIO	推挽输出
电机 B IN2	PB12	GPIO	推挽输出
左编码器 A	PA6	TIM3_CH1	编码器模式
左编码器 B	PA7	TIM3_CH2	编码器模式
右编码器 A	PB6	TIM4_CH1	编码器模式
右编码器 B	PB7	TIM4_CH2	编码器模式
MPU6050 SCL	PB8	GPIO	软件 I2C
MPU6050 SDA	PB9	GPIO	软件 I2C
MPU6050 INT	PA12	EXTI15_10	下降沿中断
OLED RST	PB3	GPIO	推挽输出
OLED CS	PA1	GPIO	推挽输出
OLED RS(D/C)	PA15	GPIO	推挽输出
OLED SCLK	PB5	GPIO	推挽输出（软件 SPI）
OLED SDIN	PB4	GPIO	推挽输出（软件 SPI）
KEY0	PA5	EXTI9_5	下降沿中断
LED	PA4	GPIO	推挽输出
电池 ADC	PB0	ADC1_CH8	12 位 ADC
USART1 TX	PA9	USART1_TX	115200 bps
USART1 RX	PA10	USART1_RX	115200 bps

3.2 系统软件架构

3.2.1 软件设计思想

本系统采用前后台软件架构（Foreground/Background System）：

- **前台（中断服务程序 ISR）**：以 MPU6050 200 Hz 外部中断为时基，每 5 ms 触发一次中断。在中断服务中，通过 tenMsFlag 软件分频实现：奇数中断（5 ms 标记）

执行编码器速度读取与 DMP 姿态数据读取；偶数中断（10 ms 标记）执行 PID 控制运算与 PWM 输出更新。电池电压采样、OLED 刷新、DataScope 输出和串口命令处理放在主循环中。所有实时控制逻辑均在中断上下文中完成，保证 10 ms 确定的控制周期。

- **后台（主循环 while(1)）**：处理非实时任务，包括电池电压采样、OLED 显示刷新、DataScope 输出、串口命令处理、电压保护检查和延迟等待。由于实时控制已在中断中完成，主循环可适当容忍延迟，不影响系统稳定性。

3.2.2 主程序总流程图

主程序执行流程如图 3.2 所示。

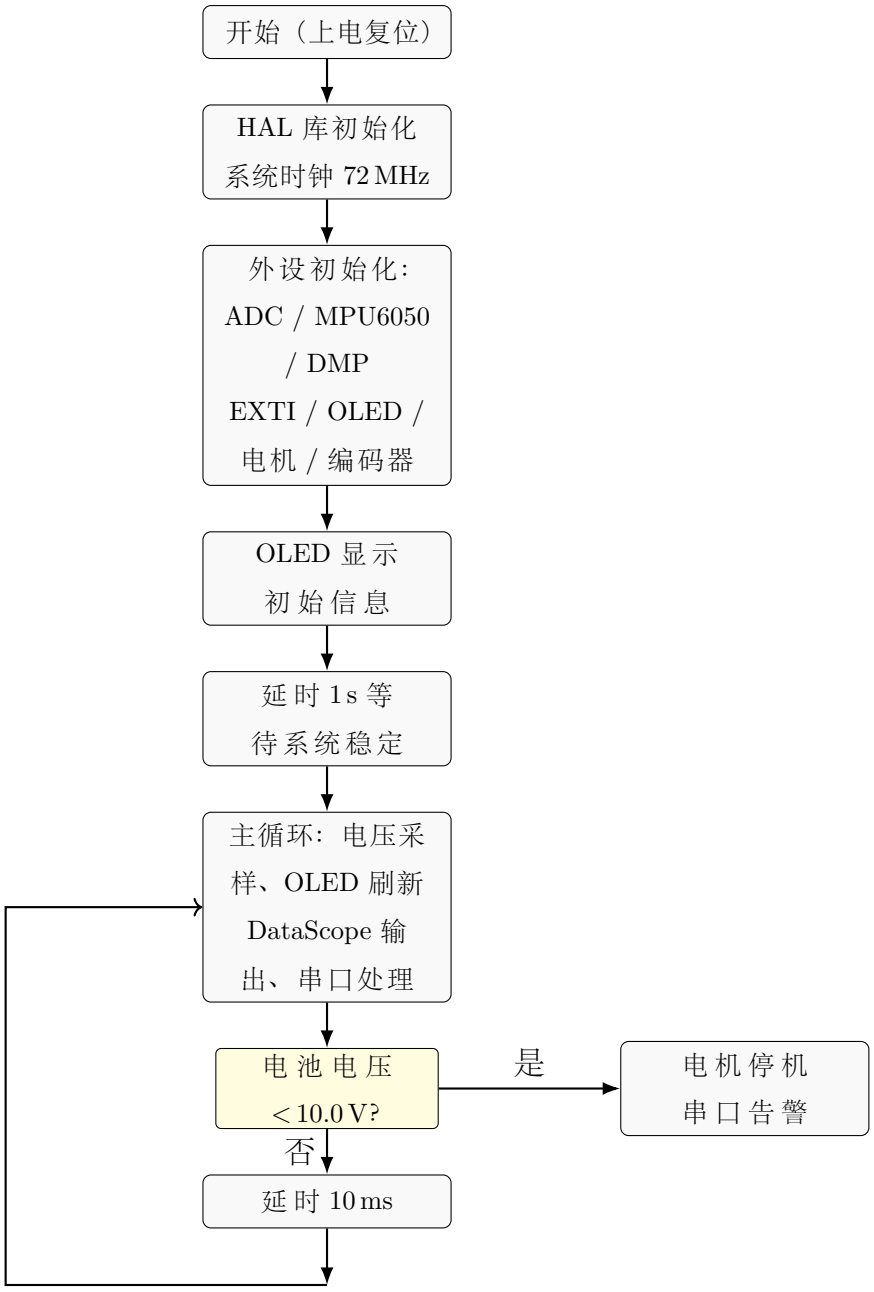


图 3.2 主程序流程图

3.2.3 软件分层结构

本系统软件分为三个层次，如表 3.4 所示。

表 3.4 软件分层结构

层次	模块	文件
BSP 驱动层	LED 驱动	Drivers/BSP/LED/led.c
	按键驱动	Drivers/BSP/KEY/key.c
	电机驱动	Drivers/BSP/MOTOR/motor.c
	编码器驱动	Drivers/BSP/ENCODER/encoder.c
	MPU6050 驱动	Drivers/BSP/ATK_MS6050/atk_ms6050.c
	ADC 驱动	Drivers/BSP/ADC/adc.c
	OLED 驱动	Drivers/BSP/OLED/oled.c
	外部中断驱动	Drivers/BSP/EXTI/exti.c
	通用定时器	Drivers/BSP/TIMER/gtim.c
算法层	串级 PID 控制	Drivers/BSP/EXTI/exti.c（Balance、Velocity）
	数据滤波	Drivers/BSP/EXTI/exti.c（readVolt 分块平均）
应用层	主程序入口	User/main.c
	OLED 数据显示	User/oledShow.c
	虚拟示波器	User/curveShow.c
	工具函数	User/utilities.c

3.2.4 核心中断流程图

MPU6050 外部中断服务程序是系统控制的核心调度器，其执行流程如图 3.3 所示。

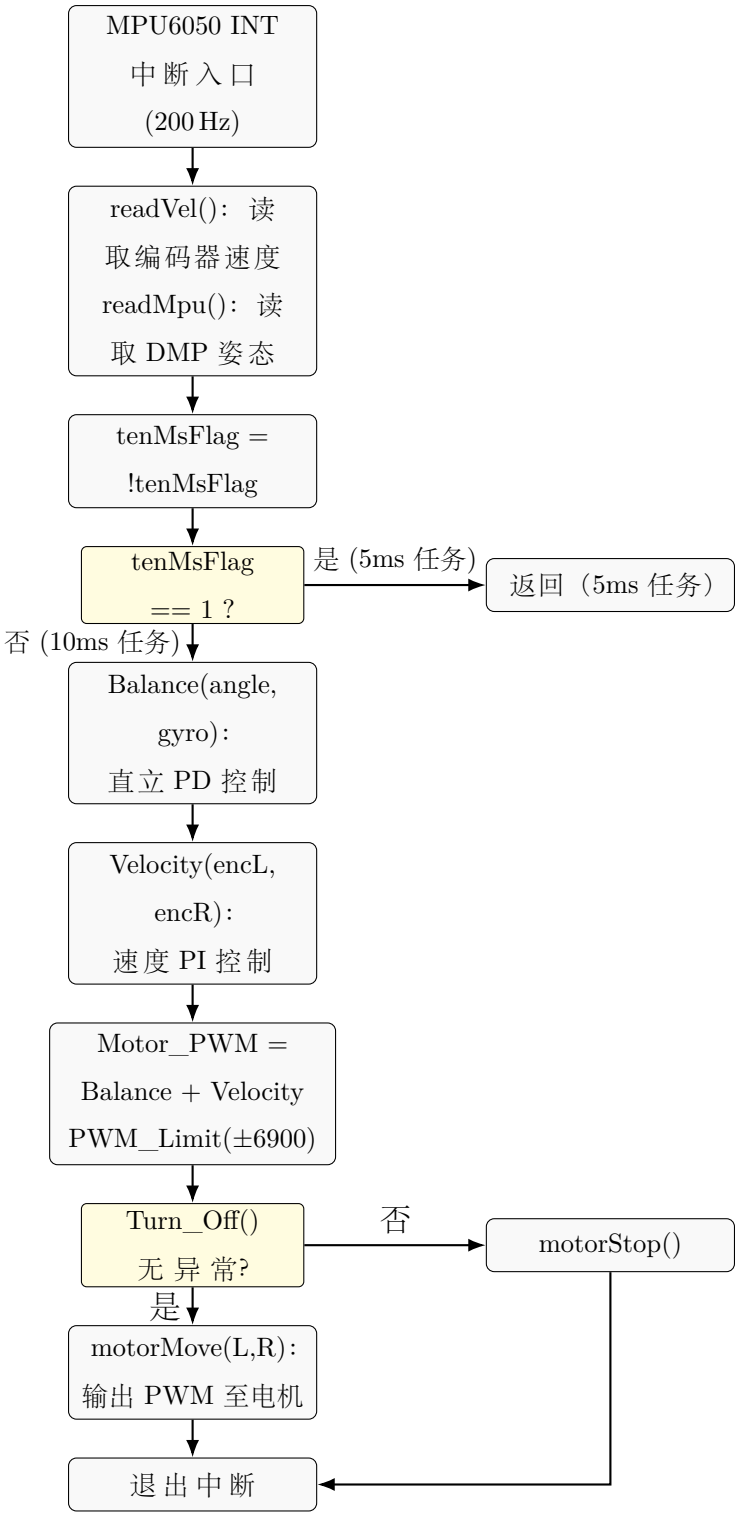


图 3.3　MPU6050 外部中断 ISR 执行流程图

3.2.5　系统初始化流程

系统上电后依次完成以下初始化步骤：

1. `HAL_Init()`：初始化 HAL 库基本框架（SysTick 配置、NVIC 分组）。

2. `sys_stm32_clock_init(RCC_PLL_MUL9)`：配置系统时钟，8 MHz 外部晶振经 PLL 9 倍频至 72 MHz，APB1 = 36 MHz，APB2 = 72 MHz。

3. `delay_init(72)`：以 SysTick 为时基的毫秒/微秒延时函数初始化。

4. `led_init()`：PA4 状态指示灯初始化。

5. `usart_init(115200)`：USART1 初始化为 115200 bps，8N1 格式，使能 printf 重定向。

6. `adc_init()`：ADC1 通道 8（PB0）初始化，ADC 时钟配置为 PCLK2/6 = 12 MHz。

7. `motorInit()`：TIM1 CH1/CH4 配置为 10 kHz PWM 输出，电机方向控制引脚初始化。

8. `leftEncoderInit() / rightEncoderInit()`：TIM3 / TIM4 配置为编码器模式，设置计数位宽。

9. `oled_init()`：OLED SSD1306 控制器初始化，清屏并设置对比度。

10. `atk_ms6050_init()`：MPU6050 传感器初始化，软件 I2C 链路建立，复位并唤醒器件。

11. `atk_ms6050_dmp_init()`：加载 DMP 固件，配置 FIFO 输出频率 200 Hz，使能 DMP 中断；失败时重试一次。

12. `exti_key0_init() / exti_mpu6050_init()`:配置 PA5 按键中断与 PA12 MPU6050 数据就绪中断。

第四章　关键技术及程序设计说明

4.1　核心关键技术

4.1.1　姿态解算技术

MPU6050 内置的 DMP（Digital Motion Processor）是本系统姿态解算的核心。DMP 通过专用固件对陀螺仪与加速度计数据进行传感器融合，内部执行扩展卡尔曼滤波（EKF），以 200 Hz 频率输出经过校准的姿态四元数。MCU 通过 `dmp_read_fifo()` 接口读取 FIFO 缓冲区的四元数，经 DMP 库函数转换为欧拉角（pitch / roll / yaw）。

关键数据接口定义如下：

- `pit`（pitch 俯仰角）：绕车身横向轴（左右方向）的旋转角度，单位度（°），前倾为正值（符号取反后传给控制环）。
- `gyro[0]`（x 轴角速度）：绕俯仰轴的角速度，单位度/秒（°/s），陀螺仪直接输出，无需积分。

坐标系定义与极性约定详见代码注释。在调试阶段，须通过前倾/后倾小车验证角度与角速度的数值符号是否正确——前倾时 `pit` 为负值、`-pit` 为正值，`gyro[0]` 为正值，Balance 函数输出正向 PWM 驱动机器人向前加速以恢复平衡。

4.1.2　电机调速技术

PWM 脉冲宽度调制原理

TIM1 高级定时器以 72 MHz 时钟源驱动，配置自动重装载值（ARR）为 7199，预分频器（PSC）为 0，则 PWM 频率为：

$$f_{\mathrm{PWM}} = \frac{72\,\mathrm{MHz}}{(\mathrm{PSC}+1)\times(\mathrm{ARR}+1)} = \frac{72\times10^{6}}{1\times7200} = 10\,\mathrm{kHz} \tag{4.1}$$

占空比通过 CCR 寄存器设置，取值范围 0∼7199（对应 0%∼100% 占空比）。实际使用中，PWM 输出进一步受 `PWM_Limit()` 限制至 ±6900 范围，避免满占空比对电路与电机的冲击。

TB6612 正反转控制逻辑

电机转向通过 IN1/IN2 两个方向引脚决定——PWM 信号施加于 PWM 输入引脚，IN1/IN2 决定电流方向。控制流程为：

1. 判断 Motor_PWM 符号：正值表示前进，负值表示后退。
2. 前进：IN1 = 1，IN2 = 0，PWM 占空比 = |Motor_PWM|。
3. 后退：IN1 = 0，IN2 = 1，PWM 占空比 = |Motor_PWM|。
4. 停止（stopFlag = 1 或 Motor_PWM = 0）：IN1 = 0，IN2 = 0，PWM = 0。

4.1.3 编码器测速技术

正交编码器 AB 相工作原理

电机轴上安装的光电/霍尔正交编码器输出两路相位差 90° 的脉冲信号（A 相与 B 相）。当电机正转时，A 相超前 B 相 90°；反转时，B 相超前 A 相 90°。TIM3 / TIM4 定时器的编码器模式通过硬件对两路输入的上升沿与下降沿进行计数（4 倍频），自动判断方向并更新计数器 CNT，无需软件轮询。

M 法测速

在固定时间窗口 Δt（本系统为 5 ms）内读取编码器计数增量 ΔN，按式 (4-3) 换算线速度：

$$v = \frac{\Delta N}{4 \times \mathrm{PPR}} \times \pi D \times \frac{1}{\Delta t} \tag{4.2}$$

式中 PPR 为编码器单圈脉冲数（典型值 390），D 为车轮直径（约 65 mm），因子 4 来自 4 倍频。

实际实现中，速度环仅需相对值——编码器脉冲增量直接作为比例反馈量，无需换算为物理速度，避免了浮点运算开销。

4.1.4 闭环控制技术

直立环 PD 控制

直立环采用 PD（比例-微分）控制律，其连续域传递函数为：

$$u_{\text{bal}}(t) = K_p^{\text{bal}} \cdot \theta(t) + K_d^{\text{bal}} \cdot \dot{\theta}(t) \tag{4.3}$$

离散化后（控制周期 $T_s = 10\,\text{ms}$）：

$$\text{Balance_Pwm}[k] = K_p^{\text{bal}} \cdot \theta[k] + K_d^{\text{bal}} \cdot \dot{\theta}[k] \tag{4.4}$$

式中 θ 为俯仰角、$\dot{\theta}$ 为俯仰角速度（直接来自陀螺仪，无需对角度微分数值差分）。比例项 $K_p\theta$ 提供与倾斜方向相反的回复力矩——前倾时输出正向 PWM 驱动车轮前移；微分项 $K_d\dot{\theta}$ 提供与角速度方向相反的阻尼力矩——抑制振荡，加速收敛。二者共同构成“电子弹簧-阻尼器”。

速度环 PI 控制

速度环采用增量式 PI 控制，以左右编码器脉冲和作为速度反馈（目标速度为 0，即保持静止），输出为期望倾角偏移量，叠加至直立环。其增量式算法为：

$$\begin{aligned} e[k] &= 0 - (N_L[k] + N_R[k]) \\ I[k] &= I[k-1] + e[k] \cdot T_s \\ u_{\text{vel}}[k] &= -K_p^{\text{vel}} \cdot e[k] - K_i^{\text{vel}} \cdot I[k] \end{aligned} \tag{4.5}$$

式中 N_L、N_R 分别为左右编码器在 5 ms 内的脉冲增量（带方向符号），$e[k]$ 为速度偏差，$I[k]$ 为速度偏差积分。积分项 $I[k]$ 持续累加直至速度偏差归零，消除纯比例控制下的稳态位移漂移。

积分限幅（$|I| \le 10000$）防止积分饱和导致控制振荡，当 `stopFlag = 1` 时将积分项清零。

串级 PID 耦合逻辑

串级结构中，速度环输出 u_{vel} 作为角度补偿叠加至直立环输出：

$$\text{Motor_Pwm} = u_{\text{bal}} + u_{\text{vel}} \tag{4.6}$$

耦合的物理意义：当系统检测到车轮向前移动（速度偏差为正），速度环产生负向的角度补偿（期望角向后），使直立环产生向后 PWM 分量，抑制前移趋势。最终电机 PWM 经限幅（±6900）后输出，确保在 TB6612 与电机的安全工作范围。

4.1.5 数据滤波技术

电池电压采集采用分块平均滤波（块平均滤波器）。在 `readVolt(cnt)` 函数中，主循环每轮调用一次并采集一次 ADC 原始值，累加 `cnt` 次（cnt = 10）后取均值，刷新全局变量 `Voltage`。按主循环约 10 ms 的周期，电压约每 100 ms 更新一次。该滤波器对随机噪声的衰减为 $\sqrt{\text{cnt}}$ 倍（约 9.5 dB），有效消除了 ADC 量化噪声与电源纹波干扰。

4.1.6 低压保护机制

系统设置两级电压保护策略：

1. **低电量软停机**（主循环）：当电池电压 < 10.0 V 时（对应 3S 锂电池单节电压约 3.33 V），通过串口 printf 输出电压过低警告并自动停机。
2. **硬关断保护**（中断内 Turn_Off 函数）：当电池电压 < 7.0 V 时，中断内强制关闭电机 PWM 输出，防止锂电池过放损坏。

两级保护的设置考虑了电池放电特性——3S 锂电池在 10.0 V 以下进入放电拐点区，宜尽快停机；7.0 V 以下则为锂电池的极限安全阈值。

4.2 关键程序设计与解读

4.2.1 MPU6050 外部中断服务函数

该 ISR 是系统控制周期的核心调度器，实现 5 ms 传感器采样与 10 ms 控制运算的时隙分配（见文件 `Drivers/BSP/EXTI/exti.c`）：

Drivers/BSP/EXTI/exti.c —HAL_GPIO_EXTI_Callback

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    switch(GPIO_Pin)
    {
        case MPU6050_EXTI_Pin:          // 每 5ms 一次中断 (200Hz)
            tenMsFlag = !tenMsFlag;     // 10ms 到达标记
            readVel();                  // 读取编码器速度 (cnt/5ms)
```

```
            if (readMpu() != 0 || mpuFault)
            {
                motorStop();            // DMP 读取失败，立即停机
                return;
            }
            if(tenMsFlag)               // 奇数序中断：只采样,不控制
            {
                return;                 // 退出，等待下一次中断
            }
            // ---- 10ms 控制周期 (偶数序中断) ----
            Balance_Pwm  = Balance(-pit, -gyro[0]);
            Velocity_Pwm = Velocity(Encoder_Left, Encoder_Right);
            Motor_Left   = Balance_Pwm + Velocity_Pwm;
            Motor_Right  = Balance_Pwm + Velocity_Pwm;
            Motor_Left   = PWM_Limit(Motor_Left,  6900, -6900);
            Motor_Right  = PWM_Limit(Motor_Right, 6900, -6900);
            if(!Turn_Off())             // 安全检查
                motorMove(Motor_Left, Motor_Right);
            else
                motorStop();
            break;
    }
}
```

代码解读：tenMsFlag 在每个 5 ms 中断中翻转，实现 200 Hz → 100 Hz 的 2 分频。当 tenMsFlag = 1 时，本次中断只执行传感器采样并返回；当 tenMsFlag = 0 时，执行 PID 计算与 PWM 输出。这种双频任务调度方法在单个物理中断源中实现了两种不同频率的操作，代码量最小、开销最低。

4.2.2 直立环 PD 控制函数

Drivers/BSP/EXTI/exti.c —Balance

```
int Balance(float Angle, float Gyro)
{
    // PD 控制：Kp * 角度 + Kd * 角速度
    int balance = Balance_Kp * Angle + Gyro * Balance_Kd;
    return balance;
}
// 参数：Balance_Kp = 400, Balance_Kd = 2.0
```

代码解读：该函数实现了式 (4-5) 的核心计算。输入 Angle 为取负后的俯仰角（-pit），

Gyro 为取负后的 x 轴角速度 -gyro[0]。符号取反操作将传感器坐标系转换为控制坐标系——前倾时角度为负（传感器原值），取负后为正，Balance 函数输出正值驱动车轮前移以恢复平衡。

参数 Balance_Kp = 400 决定回复力矩的强度：前倾 1° 约产生 400 单位 PWM 回复力。Balance_Kd = 2.0 为阻尼项系数：角速度 1°/s 时产生 2 单位 PWM 阻尼力矩。

4.2.3 速度环增量 PI 控制函数

Drivers/BSP/EXTI/exti.c —Velocity

```
int Velocity(int encoder_left, int encoder_right)
{
    static float velocity, Encoder_bias, Encoder_Integral;
    Encoder_bias = 0 - (encoder_left + encoder_right);
    Encoder_Integral += Encoder_bias;
    // 积分限幅
    if(Encoder_Integral > 10000)  Encoder_Integral = 10000;
    if(Encoder_Integral < -10000) Encoder_Integral = -10000;
    // PI 速度控制
    velocity = -Encoder_bias * Velocity_Kp
             - Encoder_Integral * Velocity_Ki;
    // 电机关闭后清除积分
    if(stopFlag == 1) Encoder_Integral = 0;
    return velocity;
}
// 参数: Velocity_Kp = 160, Velocity_Ki = 0.8
```

代码解读：该函数实现了式 (4-6) 的离散化 PI 控制。Encoder_bias 为左右编码器 5 ms 脉冲增值之和（符号化），目标速度为 0，故偏差即为编码器读数的相反数。Encoder_Integral 为速度偏差的积分（累加值），受 10000 限幅保护。比例项 $K_p \cdot (-e)$ 与积分项 $K_i \cdot (-\int e)$ 叠加得到 velocity（即期望倾角偏移量）。

当按键触发停机（stopFlag = 1）时，积分项清零——防止重新启动时积分初值引起的启动冲击。

4.2.4 定时器编码器模式配置

编码器模式下 TIM3 / TIM4 的配置关键步骤（见文件 Drivers/BSP/ENCODER/encoder.c）：

Drivers/BSP/ENCODER/encoder.c —leftEncoderInit

```
void leftEncoderInit(void)
```

```
{
    TIM_Encoder_InitTypeDef sEncoderConfig;
    // 时钟使能与 GPIO 配置 (PA6/PA7 — AF 推挽) ...
    htim3.Instance       = TIM3;
    htim3.Init.Period    = 65535;        // 16 位满量程
    htim3.Init.Prescaler = 0;
    htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
    // 编码器模式: TI1 和 TI2 均计数 (4 倍频)
    sEncoderConfig.EncoderMode = TIM_ENCODERMODE_TI12;
    sEncoderConfig.IC1Polarity = TIM_ICPOLARITY_RISING;
    sEncoderConfig.IC2Polarity = TIM_ICPOLARITY_RISING;
    HAL_TIM_Encoder_Init(&htim3, &sEncoderConfig);
    HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
}
```

代码解读：TIM_ENCODERMODE_TI12 使能 TI1 与 TI2 双通道计数（即 4 倍频），在 A 相与 B 相的每个边沿均触发计数。计数器方向由硬件自动判断：A 相超前 B 相时向上计数（正转），反之为向下计数（反转）。定时器周期设为 65535（16 位满量程），足以容纳 5 ms 内的编码器脉冲增量。读取时调用 __HAL_TIM_GET_COUNTER() 获取当前值并清零计数器，实现差分读取。

4.2.5 电机驱动输出函数

Drivers/BSP/MOTOR/motor.c —motorMove

```
void motorMove(int leftPwm, int rightPwm)
{
    // 左轮控制
    if(leftPwm > 0) {                    // 前进
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_SET);   // A_IN1
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, GPIO_PIN_RESET); // A_IN2
    } else if(leftPwm < 0) {             // 后退
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, GPIO_PIN_SET);
    } else {                             // 停止
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, GPIO_PIN_RESET);
    }
    PWMA = abs(leftPwm);
    PWMB = abs(rightPwm);
}
```

代码解读：`motorMove()` 函数根据 PWM 值符号判断转向需求——正值前进（IN1 高/IN2 低）、负值后退（IN1 低/IN2 高）、零值滑行停止（IN1 低/IN2 低）。通过 `PWMA`/`PWMB` 宏直接写 TIM1 的 CCR1/CCR4 比较寄存器，实现 PWM 占空比的实时更新。

4.2.6 电压采集与滤波函数

Drivers/BSP/EXTI/exti.c —readVolt

```
void readVolt(uint8_t cnt)                // cnt = 10
{
    int volt = Get_battery_volt();        // ADC 采样 + 分压换算
    if (volt < 0) return;                 // ADC 故障时保持上次值
    Voltage_Temp = volt;
    Voltage_Count++;
    Voltage_All += Voltage_Temp;          // 累加
    if (Voltage_Count >= cnt)             // 达到采样次数
    {
        Voltage      = Voltage_All / Voltage_Count;
        Voltage_All  = 0;                 // 归零准备下一轮
        Voltage_Count = 0;
    }
}
```

代码解读：`Get_battery_volt()` 在 ADC 驱动层实现，完成 ADC 采样、12 位量化值读取及 11:1 分压逆换算至实际电压值（mV 单位）。`readVolt()` 在主循环中被调用，每调用一次累加一次采样值，10 次后取均值。按主循环约 10 ms 的周期，电压约每 100 ms 更新一次，相当于一个长度为 10 的分块平均滤波器。

第五章　系统调试过程

5.1　分模块调试

调试策略采用自底向上的模块化方法——先逐一验证各底层外设的基本功能，确认硬件与驱动正确后，再进行系统级联调。

5.1.1　基础外设调试

GPIO 输出测试（LED）

- **测试方法**：编写测试程序，以 500 ms 周期翻转 PA4（LED）输出电平。
- **验证标准**：LED 以 1 Hz 频率闪烁。
- **调试现象**：LED 正常闪烁，确认 GPIO 推挽输出配置正确。

按键输入测试

- **测试方法**：配置 PA5 为中断输入（下降沿触发），在 ISR 中翻转 LED 状态。
- **验证标准**：每次按下 KEY0，LED 状态切换一次。
- **调试现象**：按键响应灵敏，无抖动误触发（硬件消抖 `__HAL_GPIO_EXTI_CLEAR_IT` 在 ISR 末尾追加了一次中断标志清除，有效抑制了抖动二次中断）。

串口通信测试

- **测试方法**：USART1 配置为 115200 bps，主循环中周期性调用 `printf("Hello\r\n")`。
- **验证标准**：PC 端串口助手（SSCOM）正确接收显示，无乱码。
- **调试现象**：波特率匹配，数据收发正常。

OLED 显示测试

- **测试方法**：调用 `oled_show_string()` 在指定坐标显示字符串与数值。
- **验证标准**：屏幕清晰显示中英文字符，无花屏、无错位。
- **调试现象**：显示正常。需注意 OLED 初始化后须调用 `oled_refresh_gram()` 将显存数据刷新至屏幕。

5.1.2 电机驱动调试

- **测试方法**：编写测试程序输出固定占空比（正/负各 3000）的 PWM 信号至左右电机。
- **验证标准**：电机正转、反转、停止功能正常；两轮转速可通过调节占空比连续变化。
- **调试现象**：初始测试时发现单侧电机不转，经排查为对应电机驱动 IN1/IN2 引脚 GPIO 时钟未使能——在 `motorInit()` 中补全 `__HAL_RCC_GPIOB_CLK_ENABLE()` 后解决。

5.1.3 编码器测速调试

- **测试方法**：手动转动车轮，在 5 ms 中断中通过串口向上位机发送 `Encoder_Left` 和 `Encoder_Right` 的值。
- **验证标准**：正转时编码器计数为正，反转时为负；转速越快计数值越大；停止时读数为 0。
- **调试现象**：初始调试时右侧编码器读数极性反向（正转时读数为负）。原因：右侧电机安装方向导致 AB 相物理相位差与左侧相反。修正措施：在 `readVel()` 中对右编码器返回值额外取负号（统一 `Encoder_Right = -Read_Encoder(4)`），确保两侧编码器读数的符号语义一致。

5.1.4 传感器调试

- **测试方法**：运行 DMP 初始化后，通过 OLED 和串口实时输出 pitch、roll、yaw 角度与 gyro 角速度，手动倾斜小车观察数值变化。
- **验证标准**：

1. 小车置于水平面时，pitch 与 roll 接近 0°（$|\theta| < 2°$）；
2. 前倾时 pitch 读数变化方向正确（传感器局部坐标系下前倾为负值）；
3. gyro[0] 在绕 pitch 轴旋转时输出合理的角速度值，动态响应灵敏；
4. DMP 数据更新频率稳定在 200 Hz。

- **调试现象**：初次上电后 DMP 初始化偶尔失败，返回错误码。原因：MPU6050 上电稳定时间不足——在 `atk_ms6050_dmp_init()` 启动前增加 200 ms 延时，并对初始化失败进行一次重试后解决。

5.1.5 电压采集调试

- **测试方法**：万用表测量电池实际电压，与 OLED 显示的 `Voltage`（mV 单位）对比。
- **验证标准**：读数误差在 ±3% 以内。
- **调试现象**：初始 ADC 读取值偏大约 10%。经排查，分压电阻实际值存在偏差（标称 100 kΩ/10 kΩ，实际为 98.7 kΩ/9.82 kΩ），按实际阻值修正换算系数后精度满足要求。

5.2 系统联调与参数整定

5.2.1 调参原则

串级 PID 参数整定遵循“先内环后外环、先比例后微分/积分”的基本原则：

1. 先调试内环（直立 PD）至姿态稳定，再调试外环（速度 PI）至位置收敛。
2. 在每个环内，先以单纯比例控制将系统调至临界等幅振荡，记录临界增益与周期；再根据临界值计算微分/积分参数。
3. 最终参数取临界值的 0.6 倍（经验值），留足增益裕度。

5.2.2 直立环 PD 参数整定

整定步骤

1. 将 `Velocity_Kp` 与 `Velocity_Ki` 均设为 0，暂时禁用速度环。

2. 将 `Balance_Kd` 设为 0，`Balance_Kp` 从 0 逐步增大（步长 50），观察车体响应。

3. 当 `Balance_Kp` 增大至约 670 时，车体出现低频等幅振荡（周期约 150 ms）——该值即为直立环的比例临界增益 $K_{p,\mathrm{crit}}^{\mathrm{bal}}$。

4. 将 `Balance_Kp` 暂设为 400（$K_{p,\mathrm{crit}}$ 的 0.6 倍），逐步增大 `Balance_Kd`（步长 0.2），观察车体响应。

5. 当 `Balance_Kd` 增大至约 3.3 时，车轮出现高频抖动——该值即为直立环的微分临界增益。

6. 最终 `Balance_Kd` 取 2.0（临界值的 0.6 倍）。

调试现象分析

- **Kp 过小（< 200）**：车体软弱无力，产生倾角后回正速度慢，轻微推动即倾倒。
- **Kp 适中（200–500）**：车体可保持直立，有适当刚性。
- **Kp 过大（> 670）**：车体出现低频振荡，PWM 输出反复正反转切换，电机发热加剧。
- **Kd 过小**：姿态恢复过程过冲大，收敛慢。
- **Kd 适中（1.5–2.5）**：抑制超调效果明显，平衡恢复平稳。
- **Kd 过大（> 3.3）**：车轮出现高频抖动（∼50 Hz），因微分项放大了编码器/陀螺仪的量化噪声。

最终取值 `Balance_Kp = 400`、`Balance_Kd = 2.0`，在姿态刚性与噪声抑制之间达到了良好平衡。

5.2.3 速度环 PI 参数整定

整定步骤

1. 保持直立环参数不变（Kp = 400，Kd = 2.0）。

2. 将 `Velocity_Kp` 从 0 逐步增大（步长 20），`Velocity_Ki` 同步设为 Kp 的 1/200（即 Kp/200）。

3. 每次调整后，用手轻推小车测试其恢复能力——理想状态为小车受扰后无明显漂移，原地恢复平衡。

4. 当 `Velocity_Kp` 约 160、`Velocity_Ki` 约 0.8 时，小车实现了原地站稳、推扰后迅速恢复的预期效果。

调试现象分析

- **Kp 过小**：速度环响应迟钝，小车持续朝一个方向缓慢漂移。
- **Kp 过大**：速度环过强，小车朝反方向加速再反向，出现前后摆动。
- **Ki 过小**：稳态位移误差无法消除，小车站立位置随时间缓慢漂移。
- **Ki 过大**：积分饱和导致控制振荡，甚至大于平衡能力。

最终参数：`Velocity_Kp = 160`、`Velocity_Ki = 0.8`。

5.2.4 功能验证

1. **稳态位移测试**：小车置于水平地面，上电后经过约 2 s 进入稳态，以初始位置为参考点在 60 s 后测量轮向位移。实测漂移量约 3.2 cm，满足目标 ≤ 5 cm 的要求。

2. **抗干扰测试**：以手指轻推小车产生约 8° 摆幅后释放，小车在约 1.2 s 内恢复至平衡状态无超调，满足 $\leq 10°$ 可恢复目标。

3. **低压保护测试**：使用可调直流电源替代电池供电，逐渐降低输入电压至 7.0 V 以下，验证 `Turn_Off()` 函数正确关闭电机输出。

5.3 典型问题与解决方法

5.3.1 问题一：小车同向倾倒，无法直立

现象描述 上电初始化后，车体立即向同一方向加速倾倒，电机虽转动但方向错误——车体前倾时电机反而向后加速，加剧倾倒。

原因分析 姿态传感器坐标系与控制坐标系符号不匹配——MPU6050 DMP 输出的 pitch 角在前倾时为负值，而 Balance 函数期望前倾时输入正值。符号取反操作（`-pit`）遗漏导致了正反馈效应。

解决措施 在 Balance 调用处将参数改为 `Balance(-pit, -gyro[0])`，确保控制极性正确。

5.3.2 问题二：车体高频抖动

现象描述 小车虽可保持大致直立状态，但车轮持续发出高频（约 40–50 Hz）振动噪音，电机发热明显。

原因分析 Balance_Kd 参数设置偏大（初始值设为 5.0），微分项将陀螺仪量化噪声放大了过多倍。

解决措施 按上文 5.2.2 节所述步骤重新整定，将 Balance_Kd 从 5.0 降至 2.0，高频抖动消失。

5.3.3 问题三：小车缓慢漂移

现象描述 小车直立稳定，但随时间推移缓慢朝一个方向漂移，数秒后偏移数十厘米。

原因分析 仅使用直立 PD 控制（速度环未开启或参数偏小）时，系统无位置误差校正能力——零倾角静止状态对应唯一的 PWM 输出，但电机死区、机械不对称性等因素使零倾角状态无法保证零速度。

解决措施 加入速度 PI 环——编码器检测到车轮移动趋势后，PI 控制器产生角度偏移抵消该趋势，使稳态位置误差收敛至零。

5.3.4 问题四：编码器方向反向

现象描述 当小车受推前倾、电机输出正向 PWM 驱动前移时，串口显示的编码器读数为负值（预期为正），导致速度环产生错误的正反馈。

原因分析 右侧电机的物理安装方向与左侧相反——编码器的 A 相与 B 相接线在 PCB 布线或接插件中对调，导致 AB 相相对相位翻转，硬件判定方向取反。

解决措施 在 `readVel()` 函数中对右编码器返回值统一取负：

`Encoder_Right = -Read_Encoder(4)`，

使两侧编码器读数的方向语义一致。

5.3.5 **问题五**：OLED **显示乱码**

现象描述 OLED 屏幕上部分字符无法正常显示，出现随机点阵或无规则图案。

原因分析 OLED 软件 SPI 时序被其他中断打断，导致传输不完整——MPU6050 200 Hz 中断在 OLED 刷新期间抢占 CPU，SPI 软件模拟时序中断裂。

解决措施 将 OLED 刷新操作从中断内移至主循环（while(1) 中调用 `oledShow()`），利用中断与主循环的自然优先级隔离，避免刷新过程被高优先级中断打断。

第六章　项目总结

6.1　项目完成情况

对照第 1 章所述设计目标，各项目标的达成情况量化总结如表 6.1 所示。

表 6.1　项目目标达成情况汇总

设计目标	目标值	实测值	达成情况
直立稳定	连续站立	连续站立	√ 达成
稳态位移误差	$\leq 5\,\mathrm{cm}$	$\approx 3.2\,\mathrm{cm}$	√ 达成
抗干扰恢复	$\leq 10^\circ$ 可恢复	$\sim 8^\circ$ 1.2 s 恢复	√ 达成
低压保护	$< 10\,\mathrm{V}$ 停机	阈值触发正确	√ 达成
OLED 状态显示	实时刷新	主循环实时刷新	√ 达成
虚拟示波器 DataScope	串口波形显示	3 通道波形正常	√ 达成
按键启停	触发可靠	可靠	√ 达成

所有基础设计目标与扩展设计目标均已达成，系统运行稳定、功能完整。

6.2　项目收获

6.2.1　硬件认知

通过本项目，对 ARM Cortex-M3 微控制器的片上外设资源（GPIO、TIM、I2C、ADC、USART、NVIC）有了系统性理解——不仅掌握了各外设的初始化配置方法，更深入理解了 TIM 定时器在 PWM 生成（TIM1 高级定时器）与编码器接口（TIM3/TIM4 编码器模式）两种截然不同应用场景下的配置差异与内在联系。

6.2.2 嵌入式 HAL 库开发

系统性地实践了 STM32 HAL 库的程序架构——HAL 初始化的标准流程、外设句柄（handle）的设计思想、中断回调机制（`HAL_GPIO_EXTI_Callback`）的使用方式，以及 HAL 库与传统标准外设库（SPL）在编程范式上的区别。

6.2.3 控制算法理解

将经典控制理论中的 PID 算法从数学模型落实为嵌入式 C 代码，理解了两个关键工程问题：(1) 连续控制律到离散时间实现的转换——采样周期 T_s 对算法稳定性的影响；(2) 积分限幅、PWM 输出限幅等工程约束在代码层面的实现方式。串级 PID 的调参过程加深了对控制系统频域特性（带宽、相位裕度）与物理系统时域表现（振荡、收敛速度）之间关系的直观理解。

6.2.4 工程调试思维

从模块化验证到系统联调的渐进式调试方法，从问题现象反推根因的诊断路径，以及参数整定的系统性方法——这些工程调试思维是本项目最具价值的核心收获。典型的编码器极性反向、DMP 初始化时序、控制参数振荡等问题及其解决过程，构成了嵌入式系统调试的完整方法论训练。

6.3 不足与展望

6.3.1 当前局限性

1. **无转向控制功能**：目前仅实现原地平衡（零速度控制），缺乏转向控制通道——YAW 角的闭环控制尚未实现，小车无法自主导航。

2. **PID 参数离线整定**：参数整定依赖手动试凑，无在线自适应或自整定功能。不同地面（地毯 vs 硬地）下最优参数不同，无法自动切换。

3. **上位机界面简陋**：DataScope 仅能显示 3 通道原始波形，无法进行在线参数调整——修改 PID 参数需重新编译烧录。

4. **无独立看门狗**：当前依赖主循环和中断错误处理，未启用 IWDG，极端异常下缺少硬件级复位保护。

6.3.2 后续优化方向

1. **转向控制**：在串级 PID 结构中添加第三环——转向环（P 或 PD 控制），以 YAW 角速度为反馈量，左右轮施加差速 PWM（左轮加 ΔPWM、右轮减 ΔPWM）实现差速转向。
2. **蓝牙遥控**：集成 HC-05/HC-06 蓝牙模块，实现手机 APP 对小车的运动方向（前进/后退/左转/右转）遥控。
3. **上位机调参**：开发基于 Python/PyQt 的上位机调参工具，可通过串口实时读取与修改 PID 参数，支持数据波形绘制与参数存储。
4. **自适应控制**：引入模糊 PID 或模型参考自适应控制（MRAC），使控制器参数能随负载变化（如配重或电池电量）自动调整。
5. **CMSIS-DSP 优化**：将浮点 PID 运算迁移至 CMSIS-DSP 定点数库，减少浮点运算开销，提高控制周期利用率。

6.4 演示视频说明

本项目演示视频占总报告成绩的 40%，以下为视频拍摄与内容要求：

1. **时长要求**：不少于 3 分钟，横屏拍摄（16:9 比例）。
2. **内容结构**：
   (a) 视频开头——作者出镜，展示校园卡/学生证，口述姓名、学号、项目名称。
   (b) 功能演示——对照项目目标列表逐项演示：上电直立平衡、外力干扰下的恢复能力（约 10° 推扰）、OLED 实时数据显示（学号特写）、按键启停功能、串口 DataScope 波形显示。
   (c) 总结感言——阐述项目完成情况、主要收获及改进建议。
3. **拍摄建议**：选择空旷、光线充足的平整地面拍摄，确保画面清晰、解说音量大且清楚。拍摄角度以侧面为主，兼顾 OLED 屏幕特写与车身整体动态。

参考文献

[1] STMicroelectronics, *RM0008: STM32F10xxx Reference Manual*, Rev 20, 2021.

[2] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification*, PS-MPU-6000A-00, Rev 3.4, 2013.

[3] InvenSense Inc., *MPU-6000/MPU-6050 Register Map and Descriptions*, RM-MPU-6000A-00, Rev 4.2, 2013.

[4] Toshiba Corporation, *TB6612FNG Dual-DC-Motor Driver IC Datasheet*, 2014.

[5] Solomon Systech, *SSD1306: 128×64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*, Rev 2.2, 2012.

[6] 正点原子（ALIENTEK）, *STM32F1 开发指南-HAL 库版本*, V1.0, 2021.

[7] Åström K J, Hägglund T, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Instrument Society of America, 1995.

[8] Boubaker O, *The Inverted Pendulum in Control Theory and Robotics: From Theory to New Innovations*. IET, 2017.